Genes Are Prompts in Unity — architecture brief

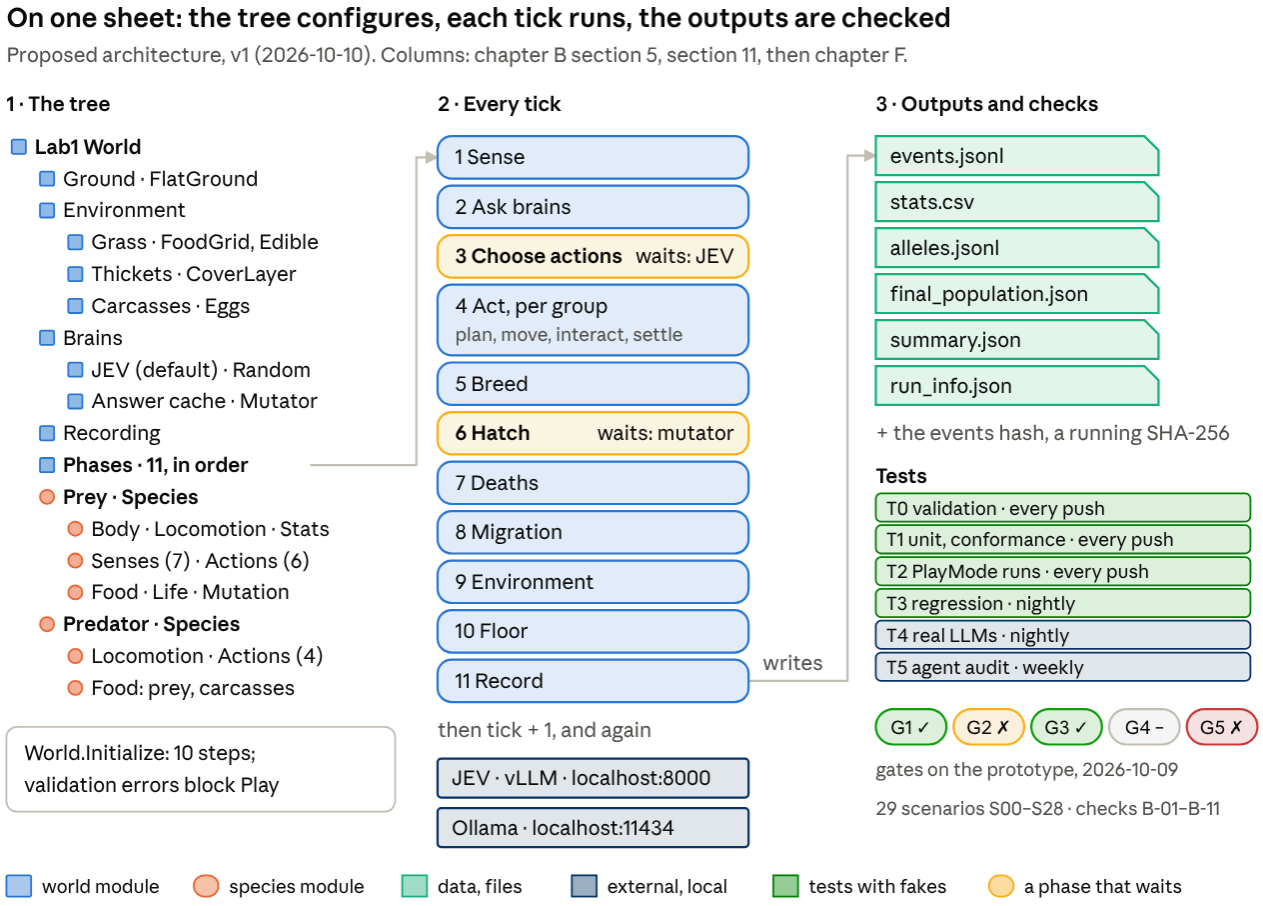
Oct 10, 2026 · @Paul

Proposed architecture, v1 (2026-10-10). A tree of replaceable components.

The system is a tree of small, replaceable components: a student who understands one base class can read, reimplement or invent one behaviour without touching the rest. Course staff build the Unity 6000.3 version from this brief; Master 2 students read it when they extend the system. Nothing is built yet, and wherever this brief and the contract in `Docs/Interface/` disagree, the contract wins.

- A. Orientation — this tab: the loop, the decisions, how to read the rest
- B. The architecture — the GameObject tree, UML diagrams, one tick, one birth, determinism
- C. Design patterns — fourteen patterns, each with where it appears and its pitfalls
- D. Custom setups — thirteen worked setups, each a list of components added or classes written
- E. Editor tooling — inspectors, windows, the validation panel, gizmos
- F. Verification — test tiers, conformance suites, scenarios, integrity checks, CI
- G. Appendix — rule ids, glossary, decisions log, open questions, what changed from Python

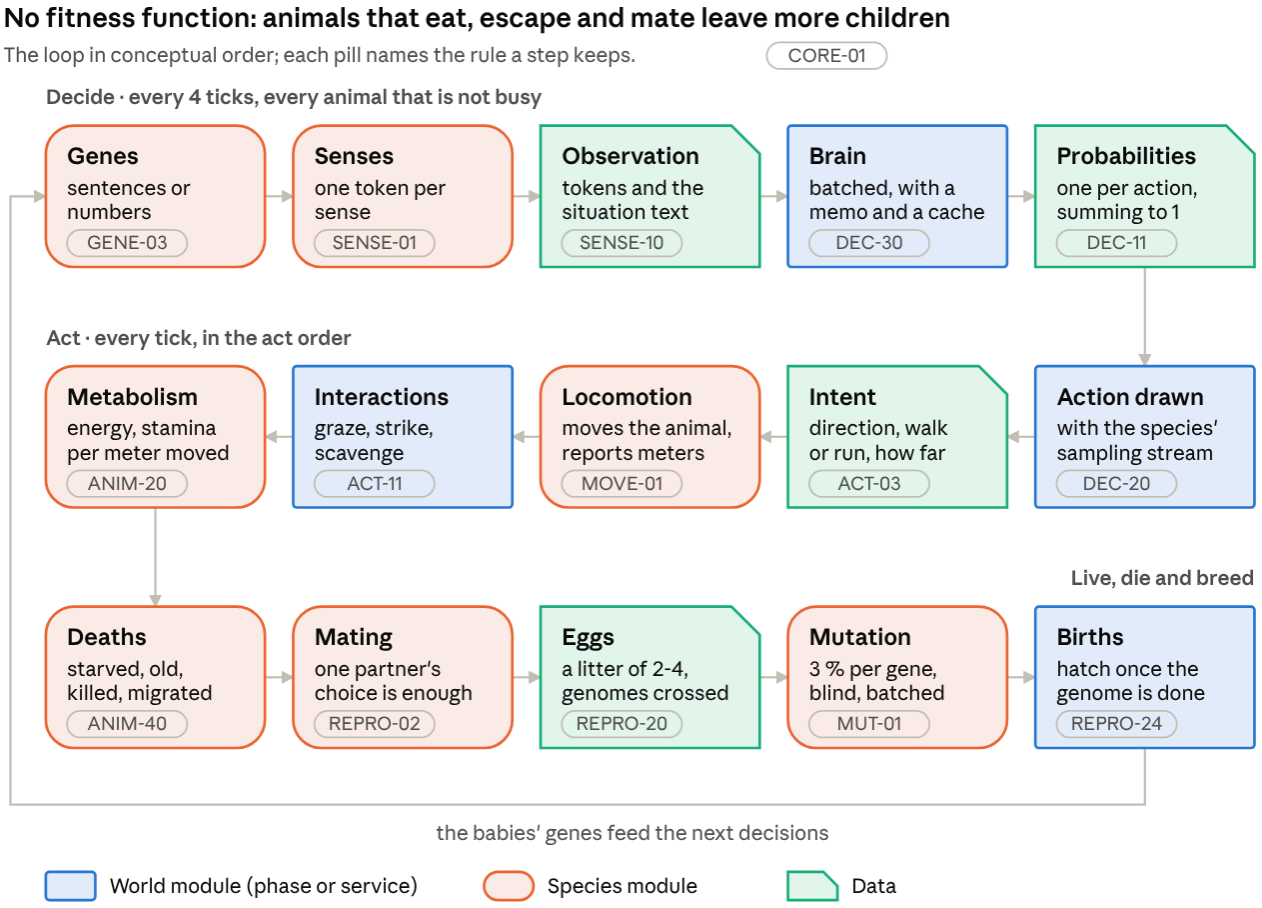
The system on one sheet



the poster · the whole system on one sheet

Left to right: the GameObject tree is the configuration, the eleven phases run it every tick, and what they record is judged by the tests, scenarios and gates. Each column has its own chapter: B for the first two, F for the third.

2. The loop



the loop · 15 steps, from genes back to genes

Read it clockwise from Genes. Ordinary code does everything physical; the brain only chooses among actions (`CORE-06`), and mutation never sees what helps (`CORE-07`). This is the conceptual order; the exact phase order of a tick is section 11 of chapter B.

3. Decisions that shape the design

The course owner took these decisions in three rounds (2026-10-09 twice, 2026-10-10). Each one is a reason the architecture looks the way it does; the full log is in chapter G.

Group	Decision	What it forces in the architecture
Space and movement	Continuous positions on a flat plane or a Unity Terrain; Euclidean distance on the horizontal plane; straight-line movement. Grids exist only inside layers (food cells, cover cells, spatial index).	`Ground` is a service; layers own their own grids; texts the brain reads say "meters".
Space and movement	One `Locomotion` component per species; walk and run speeds are its traits. Kinematic by default, no rigid bodies.	`KinematicLocomotion` is the reference; terrain, NavMesh and physics locomotion come later as subclasses.
Space and movement	An action only gives a direction (walk or run, how far at most); whatever the locomotion does is the tick's result.	Actions return an `Intent`; they never move the animal themselves.
Space and movement	The Python prototype's values, one cell read as one meter, unchanged.	Reference numbers live on component fields, not in code.
Species and food	Any number of species, not hard-coded; `prey` and `predator` in the reference. A species inside a species is an error. One `Species` class, subclassable, but composition is the main way.	Species are discovered from the tree; validator V-01.
Species and food	Edibility by components: an `Edible` on what can be eaten, a `Diet` on the eater. Eggs have no `Edible`.	The food web is derived, not declared; threats are whoever's diet strikes you.
Species and food	Cannibals list their own species and flee from their own kind.	Animal sets (threats, prey, kin) are resolved from diets, so a species can be its own threat.
Species and food	Hide and flee are separate actions, with separate genes and senses (cover).	`HideAction`, `NearestCoverSense` and `CoverLayer` are their own modules.

Group	Decision	What it forces in the architecture
Genes and mutation	Genes are an interface: text genes go into the prompt; number genes set a trait. One text gene per action by default; free genes allowed. No number genes in the reference species.	`Gene` base class with `TextGene` and `NumberGene`; the neutral allele and the founder pool live on the gene component.
Genes and mutation	Mutation is a system of its own and may be asynchronous: eggs hatch when the mutator's answer is back.	`MutationOperator` modules, a `MutatorService`, and a Hatch phase that waits.
Brains	JEV is the default brain, running locally on the owner's computer, as is the mutator.	`JevBrain` and `MutatorService` talk to local servers.
Brains	The prototype's keyword brain is not part of the Unity system.	Fast tests use `RandomBrain` and a test-only `ScriptedBrain`.
Brains	Header, per-action lines and rule lines are inspector fields with placeholders for numbers; the answer instruction is hard-coded in each brain.	Every species module can write prompt rules (`WritePromptRules`).
Brains	Observation wording is in each sense's code; labels and thresholds in the inspector. Distances only for now, angles later.	`Sense.Read` returns a token; `Sense.Write` words it.
The tick	An ordered list of phase components on the World. Species act in turn by default; all mixed and simultaneous are options (simultaneous gives contested items to a random winner).	`TickPhase` components; the act order is a strategy.
The tick	Breeding happens in its own phase after the act phase; no incubation by default.	Breed and Hatch are separate phases.

Group	Decision	What it forces in the architecture
The tick	Lockstep waiting: a decision never uses old observations. Freeze (the main thread blocks) or Responsive (the tick pauses, Unity keeps rendering), with the same results. Not real time.	`TickPhase.WaitsFor` returns a `Pending`; the run loop has Waiting and Blocked states.
The tick	Determinism per machine only; the answer cache is local JSON-lines files.	Named random streams; `AnswerCache` on disk.
Teaching	Reference modules are written exactly like student modules. Students first reimplement existing components against their tests, then invent their own.	Conformance suites per base class; the teaching path in chapter D.
Infrastructure	Unity 6000.3; Newtonsoft JSON; Unity Test Framework; CI and everything else on the owner's computer.	Version-specific code stays isolated; a self-hosted CI runner.

4. How to read this brief

Read from what must hold to how it is checked. Each step leans on the one before it:

1. **Contract** — `Docs/Interface/` 01–15: what must hold, as numbered rules (`SENSE-01` , `MOVE-04` , …). A **MUST** rule is kept by every implementation and checked by a test; a **SHOULD** rule is kept by the reference; **MAY** is optional; **Reference** is a default value or behaviour of the Lab 1 modules.
2. **Architecture** — chapter B: how the contract is built in Unity, as a GameObject tree, base classes and services.
3. **Patterns** — chapter C: the fourteen design patterns the architecture uses, and their pitfalls here.
4. **Custom setups** — chapter D: thirteen worked examples, each a list of components added or classes written.
5. **Editor** — chapter E: inspectors, windows, validators and gizmos.
6. **Verification** — chapter F: tests, scenarios, integrity checks and the CI that will judge your work.

Wherever a diagram or an example shows a rule at work, it carries the rule's id, e.g. `MOVE-04`.

Map of Docs/Interface

#	Document	Covers	Used in
01	Concepts and glossary	the loop, every term used in the other documents	A, G
02	Space and time	continuous positions, terrain, walkability, distances, ticks, waiting for the brain	B 13, B 18
03	Environment	resource layers (food), regrowth, cover, carcasses and other entities	B 9, B 17
04	Species and the food web	species, who eats whom, threats, names, species created at run time	B 17, D 37–39, D 46
05	Animals, stats and life cycle	animal state, stats, traits, metabolism, busy states, death	B 7, B 15
06	Senses and observations	discrete observations, distance bands, situation text, batched raycasts, camera senses	B 12, B 16
07	Actions and locomotion	actions, search fallback, movement, stamina budget, interactions, act order	B 11, B 18
08	Decisions, brains and prompts	when animals decide, the brain interface, the prompt, sampling, memo, cache, failures	B 12, B 16, D 43
09	Genes, alleles and genomes	text and number genes, expression, alleles, registry, founder pools, crossover	B 8, C 27, D 40
10	Mutation	rates, operators, guards, redraws, asynchronous mutation	B 14, D 44
11	Reproduction and population	mating, litters, eggs and incubation, caps, migration, floors, newcomers	B 14, B 15
12	Tick order and determinism	the phase list, act order policies, random streams, events hash, replay	B 11, B 19
13	Outputs and recording	events, stats, alleles, summary, run info	C 30

#	Document	Covers	Used in
14	Configuration reference	every parameter with its reference value and unit	D 35
15	Controls, metrics and gates	experimental controls, sensitivity metrics, gates G1–G5	F 55
20	Unity architecture	the GameObject tree, base classes, data layout, the tick loop, services, assemblies	B, C
21	Editor tooling	validation, inspectors, windows, gizmos, run controls	E
22	Extending: recipes for students	add a sense, an action, a gene, a stat, a brain, a mutation, a phase, a resource, a species	D
30	Tests	unit, regression and integration tests mapped to rule ids	F 52–53
31	Scenarios	test worlds with different options, genes, mutations, senses and actions	F 54
32	Integrity prompts and CI	brain integrity checks, coding-agent prompts, the CI tiers	F 55–56
40	Open questions	what is settled, still open or deferred	G 60